

MIDTERM DESIGN DOCUMENT

Single AI Agent: Paris Travel Planner

Deep Learning Applications | Option A | Spring 2025

1. Problem Statement

Planning a trip to Paris is a genuinely complex task. Travelers must simultaneously juggle dozens of variables: comparing hotels by location and price, cross-referencing attraction opening hours, securing hard-to-get reservations at popular restaurants or museums, and constructing a walking or transit route that is both logical and time-efficient. The typical approach, cycling through Booking.com, Google Maps, TripAdvisor, and blog posts is fragmented, time-consuming, and prone to conflicts such as scheduling a museum visit on its weekly closing day.

We propose an AI agent that acts as an intelligent personal travel advisor for Paris. Given a set of user inputs — travel dates, budget, preferences, and constraints that the agent autonomously retrieves real-time data, reasons over conflicting options, and produces a complete, day-by-day itinerary tailored to the individual traveler.

Who Benefits

- Leisure travelers who want a high-quality itinerary without spending hours researching
- First-time visitors to Paris are overwhelmed by the city's density of attractions and logistics
- Budget-conscious travelers who need to balance cost, distance, and experience quality
- Travelers with specific constraints such as dietary needs, limited mobility, or tight time windows

2. Option Choice — Why Option A (Single Agent)

We selected Option A: a single AI agent. The core task, end-to-end Paris trip planning, is a unified workflow with one clear goal: produce a feasible, personalized itinerary. A single well-structured agent with modular tool integrations is well-suited to this problem and avoids the added coordination overhead of multi-agent systems.

The reasoning and tool-use responsibilities (parsing preferences, querying APIs, optimizing routes, detecting conflicts) are tightly coupled and share the same context window, making a single-agent design more coherent. Should complexity grow beyond this scope, for example, adding a dedicated booking sub-agent or a separate user-profile personalization layer, we could scale to Option B without redesigning the core architecture.

3. Agent Architecture

The diagram below illustrates the full agent pipeline: user inputs flow into the LLM reasoning core, which orchestrates a series of tool calls, applies deep learning-driven optimization, and generates a structured output. All components are labeled.

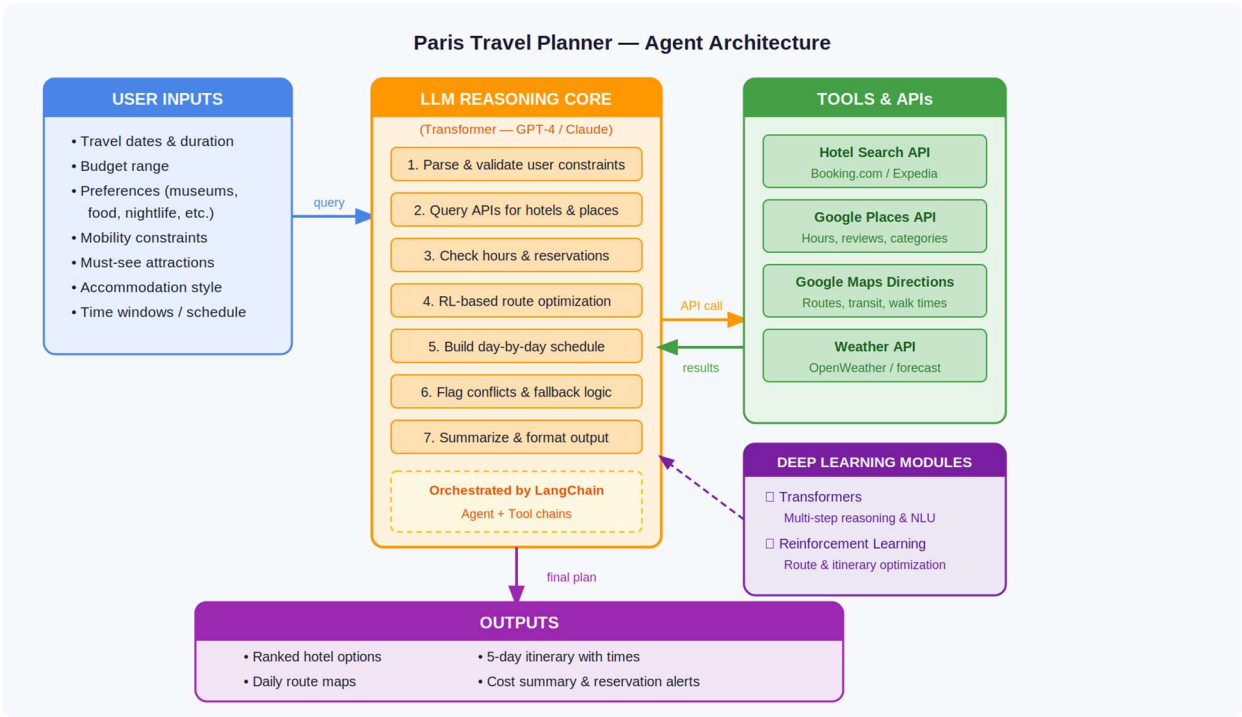


Figure 1. Paris Travel Planner — Agent Architecture Diagram

Architecture Summary

The agent operates in four logical layers:

- Inputs:** User Inputs — Travel dates, budget, preferences (food, culture, nightlife), mobility constraints, and must-see attractions are collected at the start of each session.
- Reasoning:** LLM Reasoning Core — A transformer-based LLM (GPT-4 or equivalent) serves as the central orchestrator. It interprets user intent, decomposes the planning task into steps, issues tool calls, evaluates returned data, and applies RL-informed optimization to build the final schedule.
- Tools:** Tools & APIs — The agent calls a hotel search API, Google Places (attraction hours and categories), Google Maps Directions (routes and transit times), and a weather API. Results feed back into the reasoning core for constraint evaluation.
- Outputs:** Outputs — The agent delivers ranked hotel options, a complete 5-day itinerary with arrival times, route suggestions for each day, reservation flags, and a total cost estimate.

4. Deep Learning Connection

This agent draws on two core deep learning modules from the course: Transformers and Reinforcement Learning. Both are architecturally integral, not optional additions to how the agent reasons and optimizes.

Module 1: Transformers (LLMs)

The agent's central reasoning engine is a transformer-based large language model. Transformers, introduced in the "Attention Is All You Need" architecture, use self-attention mechanisms to model long-range dependencies across an entire input sequence simultaneously, without the sequential bottleneck of RNNs. This property makes them especially powerful for multi-step planning tasks where early constraints (e.g., "I

arrive Tuesday at 3pm") must influence decisions made many reasoning steps later (e.g., "don't schedule the Eiffel Tower summit for Day 1 evening").

In this agent, the transformer LLM performs several distinct functions that rely directly on its architecture:

- Natural language understanding — parsing free-text preferences and constraints from the user into structured parameters that can drive API queries
- Multi-step constraint satisfaction — maintaining awareness of all user constraints across a chain of tool calls and reasoning steps without losing context
- Itinerary generation — producing a coherent, readable day-by-day schedule that integrates dozens of retrieved facts into a single narrative output
- Conflict detection and fallback reasoning — identifying when two scheduled items conflict (e.g., a museum is closed on the planned day) and proposing alternatives

Course Connection: The transformer module directly enables the agent's language understanding and multi-step planning capabilities. Without attention-based context retention, the agent could not hold the full constraint space in mind while generating a 5-day itinerary.

Module 2: Reinforcement Learning (Policy Optimization)

Reinforcement Learning is applied to the route and itinerary optimization sub-task — specifically, determining the best ordering of daily attractions to minimize travel time, maximize preference alignment, and stay within budget. This is modeled as a Markov Decision Process with the following components:

MDP Component	Definition in This Agent
State (S)	Current itinerary state: visited attractions, time remaining in the day, current location, remaining budget
Action (A)	Select the next attraction or hotel to add to the schedule; choose between transit options
Reward (R)	Positive: preference match, feasibility, budget adherence. Negative: excessive travel time, constraint violations, missed reservations
Policy (π)	The learned strategy that maps the current itinerary state to the optimal next scheduling decision

In practice, the agent uses a policy trained (or fine-tuned via RLHF) to maximize user satisfaction as a reward signal. During inference, rather than brute-force searching all possible itinerary orderings, the RL policy efficiently approximates the optimal sequence through learned value estimation. This is analogous to the explore/exploit trade-off studied in the course lab: the agent balances exploring new attraction combinations against exploiting arrangements known to score highly based on the user's stated preferences.

5. Agent Framework

We will use Lang Chain as our agent orchestration framework.

Why Lang Chain

- Single-agent suitability — Lang Chain’s Agent Executor is purpose-built for single-agent workflows with tool use, making it a natural fit for Option A
- Tool integration — Lang Chain provides built-in connectors for REST APIs, Google APIs, and custom Python functions, reducing boilerplate for hotel and maps integrations
- Prompt management — Lang Chain’s prompt templates and chains allow structured, reproducible reasoning steps across the planning pipeline
- LLM flexibility — the framework supports multiple LLM backends (OpenAI, Anthropic, local models), enabling provider switching without restructuring the agent
- Active ecosystem — extensive documentation, community examples, and maintained integrations reduce development risk for a short build timeline

Lang Chain’s Re-Act (Reason + Act) agent pattern is particularly well-suited here: the agent alternates between reasoning about what to do next and executing a tool call, building the itinerary incrementally rather than in a single pass.

6. Tools & Data

External APIs

- Hotel Search API (Booking.com / Expedia API or mock data) retrieves available hotels by date, location, price, and amenity filters
- Google Places API — provides attraction details including opening hours, admission fees, reservation requirements, user ratings, and category tags
- Google Maps Directions API — calculates walking, transit, and driving routes between locations including real-time travel time estimates
- OpenWeatherMap API — retrieves 5-day forecasts to flag weather-sensitive outdoor activities

Internal Data & State

- User profile — travel dates, budget ceiling, preference weights (culture, food, nightlife, outdoors), mobility constraints
- Session state — running itinerary, visited attractions list, remaining daily budget, current location
- Attraction metadata — curated dataset of Paris points of interest with pre-tagged categories, average visit durations, and known reservation windows
- Metro / transit data — Paris RATP line data for accurate transit time calculation between arrondissements

7. Build Plan

The following timeline covers the four weeks from midterm submission to the final due date of May 3.

Week	Focus	Key Deliverables
Week 1 Apr 5–11	Foundation	Set up Lang Chain project structure. Implement hotel search tool. Build basic agent loop with prompt templates. Validate API connections.

Week 2 Apr 12–18	Core Tools	Integrate Google Places and Maps APIs. Build itinerary generator with constraint checking. Test with static Paris attraction dataset.
Week 3 Apr 19–25	Optimization & Polish	Implement RL-based route optimizer. Add weather integration. Tune preference weighting. End-to-end testing with real Paris scenarios.
Week 4 Apr 26–May 3	Finalization	Add error handling and fallback logic. Polish output formatting. Write final report. Record demo video.

8. Anticipated Challenges

The following risks have been identified along with planned mitigation strategies:

Challenge	Mitigation Strategy
API rate limits on Google Places / Maps	Implement response caching and batch requests; use mock data as fallback during development
LLM hallucinations (fabricated hours, fake reservations)	Validate all LLM-generated facts against ground-truth API responses before including in the final output
Route feasibility (transit delays, unrealistic walk times)	Apply conservative time buffers between activities; use Maps API real-time data rather than estimated distances
Overlapping time windows and scheduling conflicts	Build a constraint-checking layer that runs after each itinerary draft and flags violations before finalizing output
Budget constraints vs. location preferences	Surface trade -off options to the user (e.g., "this hotel is 20% over budget but saves 40 min/day in transit") rather than silently dropping preferences
RL policy training complexity within timeline	Use a lightweight Q-learning or heuristic-guided search as the optimization baseline; full policy training is a stretch goal